



การรับและจัดการข้อมูลจากฟอร์ม (PART1)

030513702 : Selected Topics in Software Development
Asst. Prof. Dr. Phollakrit Wongsantisuk

ลักษณะของ HTML Form

ฟอร์ม (Form) ใช้กำหนดขอบเขตในการรับข้อมูลจากผู้ใช้อินพุตชนิดต่างๆ เพื่อส่งไปประมวลผลที่ฝั่งเซิร์ฟเวอร์ ซึ่งตามข้อกำหนดของ HTML รูปแบบของฟอร์มจะเป็นดังนี้

```
<form ...>
```

อินพุตชนิดต่างๆ

```
</form>
```

เราใช้แท็ก `<form>` และ `</form>` เพื่อกำหนดขอบเขตของฟอร์ม มีอินพุตหลายตัวให้เลือกเพื่อวัตถุประสงค์ที่แตกต่างกัน เช่น `text` สำหรับรับข้อมูลข้อความ หรือ `select` ใช้แสดงข้อมูลแบบตัวเลือก เป็นต้น ซึ่งสามารถสรุปได้ในตารางต่อไปนี้

ลักษณะของ HTML Form

<code><input type="text"></code>	ช่องรับข้อความแบบบรรทัดเดียว
<code><input type="password"></code>	ช่องรับข้อมูลที่เป็นรหัสผ่าน
<code><input type="checkbox"></code>	กำหนดตัวเลือกที่สามารถเลือกได้มากกว่า 1 รายการ
<code><input type="radio"></code>	กำหนดตัวเลือกที่เลือกได้เพียงรายการเดียว
<code><input type="file"></code>	กำหนดไฟล์ที่จะอัปโหลด
<code><input type="submit"></code>	ปุ่มสำหรับส่งข้อมูลจากฟอร์มไปยังเพจปลายทางที่เซิร์ฟเวอร์
<code><input type="reset"></code>	ปุ่มสำหรับยกเลิกข้อมูลทั้งหมดในฟอร์ม
<code><input type="hidden"></code>	ใช้สำหรับเก็บซ่อนข้อมูลที่ไม่ต้องการแสดงให้ผู้ใช้เห็น
<code><input type="image"></code>	เมื่อต้องการใช้รูปภาพเป็นปุ่มสั่งงาน

ลักษณะของ HTML Form

<code><input type="button"></code>	ปุ่มสั่งงานทั่วไป ส่วนใหญ่จะใช้ร่วมกับ JavaScript
<code><button type="..."></code>	เป็นปุ่มอีกแบบหนึ่ง โดย type ที่กำหนดได้ เช่น submit, reset
<code><textarea></code>	ช่องรับข้อความแบบหลายบรรทัด
<code><select></code>	สำหรับสร้างรายการตัวเลือกทั้งแบบ dropdown และแบบ list
<code><option></code>	กำหนดตัวเลือกของ Select
<code><input type="number"></code>	สำหรับการรับข้อมูลตัวเลขจำนวนเต็ม
<code><input type="email"></code>	สำหรับการรับข้อมูลอีเมล
<code><input type="url"></code>	สำหรับการรับข้อมูล URL
<code><input type="date"></code>	สำหรับข้อมูลที่เป็นวัน/เดือน/ปี ในรูปแบบ : ปี-เดือน-วันที่ เช่น 2020-10-20
<code><input type="time"></code>	สำหรับข้อมูลที่เป็นเวลา เช่น 12:34:56

ลักษณะของ HTML Form

อินพุตแต่ละชนิด ยังมีแอททริบิวต์ที่ต้องกำหนดเพิ่มเติมอีกบางส่วน โดยแอททริบิวต์บางอันอาจนำไปใช้กับอินพุตทุกชนิด แต่บางอันก็อาจใช้กับอินพุตเพียงบางชนิด ซึ่งแอททริบิวต์ที่สำคัญสามารถสรุปได้ดังตารางต่อไปนี้

type	เพื่อกำหนดชนิดของอินพุต
name	เป็นชื่อสำหรับอ้างอิงทางฝั่งเซิร์ฟเวอร์
id	เป็นชื่อสำหรับอ้างอิงด้วย JavaScript หรือ CSS
value	กำหนดค่าล่วงหน้าให้กับอินพุตนั้น (สามารถแก้ไขในภายหลังได้)
placeholder	ข้อความแบบลายน้้าบนอินพุตนั้น (ข้อความจะหายไปเมื่อใส่ข้อมูล)
maxlength	ความยาวหรือจำนวนอักขระสูงสุดที่ใส่ลงไปได้

ลักษณะของ HTML Form

min, max	สำหรับกำหนดค่าต่ำสุดและสูงสุดของอินพุต number
rows, cols	จำนวนแถวและคอลัมน์สำหรับอินพุต textarea
checked	กำหนดให้เลือกรายการนั้นสำหรับ radio และ checkbox
selected	กำหนดให้เลือกรายการนั้นสำหรับ select
readonly	อ่านได้อย่างเดียว (ไม่ต้องกำหนดค่า)
disabled	ให้อินพุตนั้น ใช้งานไม่ได้ (ไม่ต้องกำหนดค่า)
required	จะต้องใส่หรือเลือกอินพุตนั้นเสมอ (ไม่ต้องกำหนดค่า)
multiple	สำหรับอินพุต Select ให้สามารถเลือกได้หลายรายการ

การสร้างฟอร์มใน React Application

เนื่องจากใน React เราต้องเขียนโค้ด HTML ในรูปแบบของ JSX และในกรณีของฟอร์มก็เช่นเดียวกัน โดยเราอาจสร้างฟอร์มไว้ในคอมโพเนนต์ จากนั้นก็นำไปแสดงผลที่ App.js ซึ่งมีแนวทางในเบื้องต้นดังต่อไปนี้

react/app1/src/form-search.js

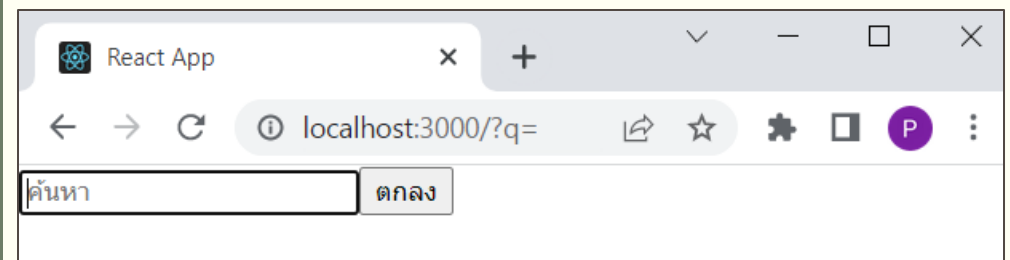
```
import React from 'react'

export default function FormSearch() {
  return (
    <form>
      <input type="text" name="q" placeholder="ค้นหา"/>
      <button>ตกลง</button>
    </form>
  )
}
```

react/app1/src/App.js

```
import React from 'react'
import FormSearch from './form-search';

export default function App() {
  return <FormSearch/>
}
```



การสร้างฟอร์มใน React Application

การเข้าถึงอินพุตภายในฟอร์ม

หากเป็นรูปแบบปกติของ JavaScript การเข้าถึงอิลิเมนต์ก็อาจใช้เมธอดประเภท `getElementXXX()` หรือ `querySelector()` แต่ใน React เราไม่ควรใช้เมธอดเหล่านั้น โดยให้ใช้วิธีการสร้างชื่ออ้างอิง (Reference) กับ อิลิเมนต์เป้าหมาย ซึ่งในกรณีที่เป็นอินพุตของฟอร์มก็ทำเช่นเดียวกับอิลิเมนต์ทั่วไป เช่น

```
function FormSearch() {  
  const keyword = React.useRef()  
  return (  
    <form>  
      <input type='text' name='q' placeholder='ค้นหา' ref={keyword}/>  
      <button>ตกลง</button>  
    </form>  
  )  
}
```

การสร้างฟอร์มใน React Application

```
class FormPerson extends React.Component {  
  const fname = React.createRef()  
  const lname = React.createRef()  
  const form = React.createRef()    //ถ้าต้องการอ้างอิงถึงฟอร์ม  
  
  render() {  
    return (  
      <form ref={this.form}>  
        <input type="text" name="fname" ref={this.fname}/><br/>  
        <input type="text" name="lname" ref={this.lname}/><br/>  
        <button>OK</button>  
      </form>  
    )  
  }  
}
```

การสร้างฟอร์มใน React Application

การกำหนดอีเวนต์สำหรับอินพุตของฟอร์ม

ที่ผ่านมาเรามักจะกำหนดอีเวนต์เฉพาะที่เกิดกับปุ่ม (button) เท่านั้น แต่จริงๆ แล้วอินพุตส่วนใหญ่ของฟอร์มสามารถเกิดอีเวนต์ได้ ซึ่งอันที่สำคัญและมีโอกาสใช้งานบ่อยๆ สามารถสรุปได้ดังนี้

ชนิดของอินพุต	อีเวนต์ที่น่าสนใจ
button, submit, reset	● click เมื่อคลิกปุ่ม
text, password, textarea, number, email, url, ...	● input เมื่อข้อความเปลี่ยนแปลงไปจากเดิม ● keypress เมื่อพิมพ์อักขระ ● keydown เมื่อพิมพ์อักขระ (ตรวจสอบอักขระได้) ● focus เมื่อย้ายโฟกัสเข้ามาในอินพุต ● blur เมื่อย้ายโฟกัสออกไปจากอินพุต

การสร้างฟอร์มใน React Application

ชนิดของอินพุต	อีเวนต์ที่น่าสนใจ
radio, checkbox	● change เมื่อเปลี่ยนสถานะของการเลือก
select	● change เมื่อเลือกรายการใหม่ (รายการที่ถูกเลือกเปลี่ยนจากเดิม)
file	● change เมื่อเลือกไฟล์
form	● submit เมื่อกำลังจะส่งข้อมูลออกไปจากฟอร์ม ● reset เมื่อ clear ข้อมูลในฟอร์ม (เช่น คลิกปุ่ม reset)

การสร้างฟอร์มใน React Application

react/app1/src/form-search.js

```
import React from 'react'

export default function FormSearch() {
  const textKw = React.useRef()
  const btOk = React.createRef()

  const onChangeKw = () => {
    if (textKw.current.value.trim() !== '') {
      btOk.current.disabled = false
    } else {
      btOk.current.disabled = true
    }
  }
}
```

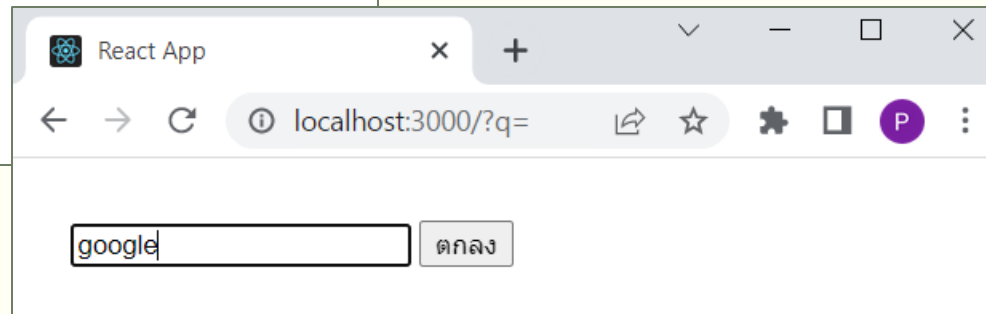
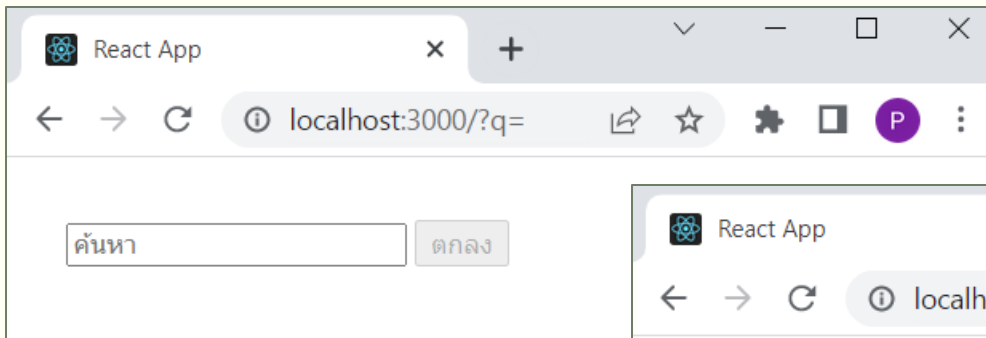
```
return (
  <div style={{margin:'30px'}}>
    <form>
      <input type="text" name="kw" placeholder="ค้นหา"
        ref={textKw} onChange={onChangeKw}/>&nbsp;
      <button ref={btOk} disabled>ตกลง</button>
    </form>
  </div>
)
```

การสร้างฟอร์มใน React Application

react/app1/src/app.js

```
import React from 'react'
import FormSearch from './form-search';

export default function App() {
  return <FormSearch/>
}
```



การใช้ Bootstrap ร่วมกับ React

การติดตั้ง Bootstrap ลงใน React Application

Bootstrap เป็นเฟรมเวิร์กที่ต้องใช้ทั้ง CSS และ JavaScript ร่วมกัน โดยไฟล์ที่จำเป็นสำหรับการทำงานของ Bootstrap มีดังนี้

- bootstrap.min.css
- bootstrap.min.js หรือ bootstrap.bundle.min.js (ไฟล์ bundle จะมีไลบรารี Popper รวมอยู่ด้วย สำหรับแสดงรายการแบบ Dropdown)

Bootstrap 5.0 ขึ้นไป ไม่จำเป็นต้องใช้ JQuery ร่วมด้วยเหมือนกับเวอร์ชันก่อนๆ อีกต่อไป ส่วนการนำไฟล์ Bootstrap มาใช้ร่วมกับ React สามารถทำได้หลายวิธี แต่โดยทั่วไปมักจะติดตั้งไฟล์ให้เป็นส่วนหนึ่งของแอป ซึ่งมีขั้นตอนดังนี้

1. ที่ Terminal ให้เข้าไปในโฟลเดอร์ของแอปที่เราต้องการติดตั้ง Bootstrap
2. พิมพ์คำสั่งแบบใดแบบหนึ่งดังต่อไปนี้ลงบน Terminal

การใช้ Bootstrap ร่วมกับ React

```
npm install bootstrap
```

```
npm install bootstrap@เวอร์ชัน
```

```
npm install bootstrap@next
```

ถ้าต้องการติดตั้งเวอร์ชันล่าสุดที่เสถียร

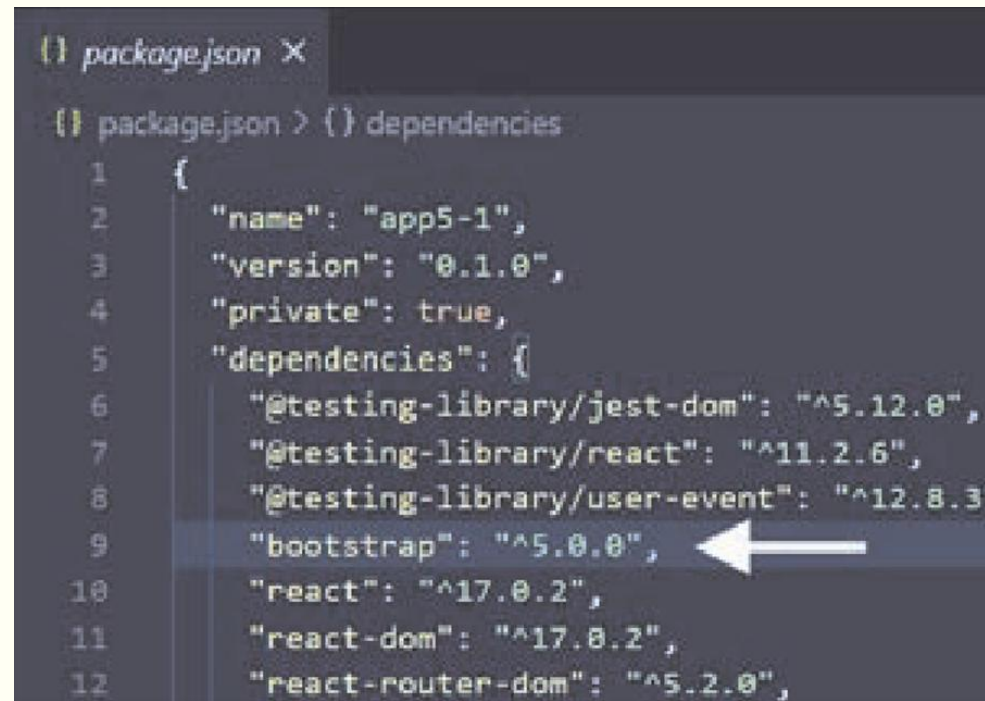
ถ้าต้องการติดตั้งเวอร์ชันตามที่ระบุ

ถ้าต้องการติดตั้งเวอร์ชันถัดไป (เช่น Beta)

3. หลังการติดตั้ง เราสามารถตรวจสอบเวอร์ชันของ Bootstrap ได้โดยเปิดดูที่ไฟล์ package.json
4. ไฟล์ทั้งหมดของ Bootstrap จะเป็นส่วนหนึ่งของแอป โดยถูกจัดเก็บไว้ในโฟลเดอร์ `node_modules`

แม้จะมีไฟล์ค่อนข้างมาก แต่ที่เราจำเป็นต้องใช้งานจริงๆ

จะมีเพียง 2 ไฟล์คือ `bootstrap.min.css` และ `bootstrap.bundle.min.js` ดังที่กล่าวไปแล้ว



```
{ package.json X
{ package.json > {} dependencies
1  {
2    "name": "app5-1",
3    "version": "0.1.0",
4    "private": true,
5    "dependencies": {
6      "@testing-library/jest-dom": "^5.12.0",
7      "@testing-library/react": "^11.2.6",
8      "@testing-library/user-event": "^12.8.3",
9      "bootstrap": "^5.0.0",
10     "react": "^17.0.2",
11     "react-dom": "^17.0.2",
12     "react-router-dom": "^5.2.0",
```

การใช้ Bootstrap ร่วมกับ React

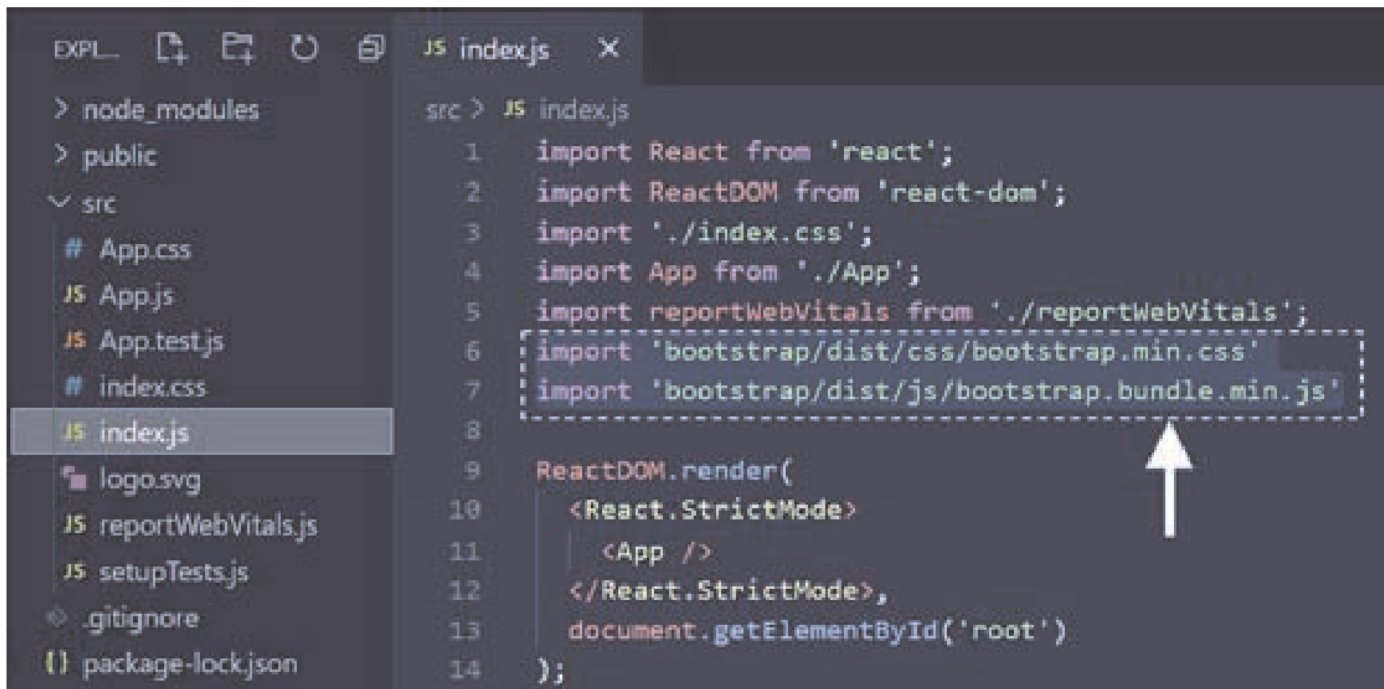
การนำไฟล์ของ Bootstrap มาใช้ใน React

หลังจากที่เราติดตั้ง Bootstrap ลงในแอปแล้ว ลำดับต่อไปจะต้องนำเข้าไฟล์ที่จำเป็นเข้ามาในแอป เหมือนกับโมดูลอื่นๆ ที่เราเคยใช้กันมา โดยไฟล์ที่เราจะนำมาใช้งานต้องอ้างอิงจากตำแหน่งภายในโฟลเดอร์ `node_modules` นั่นคือ

- `bootstrap/dist/css/bootstrap.min.css`
- `bootstrap/dist/js/bootstrap.bundle.min.js`

ตามหลักการของ React นั้น องค์ประกอบย่อยๆ เช่น คอมโพเนนต์ จะถูกนำมารวมเข้าด้วยกันในขั้นตอนสุดท้ายที่ไฟล์ `src/index.js` ก่อนจะถูกส่งไปแสดงผลยังเบราว์เซอร์ ดังนั้น ตำแหน่งที่น่าจะเหมาะสมที่สุดในการนำเข้าไฟล์ของ Bootstrap เข้ามาในแอปก็คือไฟล์ `src/index.js` เพียงแห่งเดียว โดยเพิ่มคำสั่ง `import` ลงไปในไฟล์ `src/index.js` ดังภาพ

การใช้ Bootstrap ร่วมกับ React



```
EXPL  JS index.js X
> node_modules
> public
  src
    # App.css
    JS App.js
    JS App.test.js
    # index.css
    JS index.js
    logo.svg
    JS reportWebVitals.js
    JS setupTests.js
    .gitignore
    package-lock.json

src > JS index.js
1  import React from 'react';
2  import ReactDOM from 'react-dom';
3  import './index.css';
4  import App from './App';
5  import reportWebVitals from './reportWebVitals';
6  import 'bootstrap/dist/css/bootstrap.min.css';
7  import 'bootstrap/dist/js/bootstrap.bundle.min.js';
8
9  ReactDOM.render(
10   <React.StrictMode>
11     <App />
12   </React.StrictMode>,
13   document.getElementById('root')
14 );
```

หากเรานำเข้าโมดูลของ Bootstrap มาที่ไฟล์ src/index.js ดังภาพข้างบน ต่อไปก็ไม่จำเป็นต้องนำเข้าที่ไฟล์อื่นได้อีก โดยในคอมโพเนนต์ต่างๆ เราสามารถแทรกคลาสของ Bootstrap ลงในแท็ก JSX ได้ตามต้องการ เพราะสุดท้ายแล้ว คอมโพเนนต์เหล่านั้นก็ต้องส่งมารวมกันที่ src/index.js อยู่แล้ว ซึ่งเมื่อส่งไปยังเบราว์เซอร์ คลาสของ Bootstrap ก็จะถูกประมวลผลที่นั่น ดังนั้น การนำเข้าไฟล์ของ Bootstrap ที่ไฟล์ src/index.js เพียงแห่งเดียว ก็จะมีผลต่อทุกเพจ (คอมโพเนนต์) ที่อยู่ในแอป

การปรับแต่งฟอร์มด้วย Bootstrap

- คลาสพื้นฐานของ Bootstrap ที่ใช้ร่วมกับอินพุตฟอร์ม มีดังนี้

form-group	สำหรับแบ่งกลุ่มอินพุตออกเป็นคู่ที่ประกอบด้วย label และอินพุตอันใดอันหนึ่ง
form-control	สำหรับอินพุตชนิดข้อความ
form-check	สำหรับแบ่งกลุ่มอินพุตออกเป็นคู่ที่ประกอบด้วย label และ checkbox หรือ radio (ใช้แทน form-group)
form-check-input	สำหรับอินพุตชนิด checkbox และ radio
form-check-label	สำหรับอินพุตชนิด label เฉพาะกรณีที่ใช้ร่วมกับ checkbox หรือ radio
form-check-inline	ใช้ในกรณีที่เรต้องการจัดเรียง checkbox หรือ radio ในบรรทัดเดียวกัน
form-select	สำหรับอินพุตชนิด select
form-control-file	สำหรับอินพุตชนิด file (รูปร่างในแต่ละเบราว์เซอร์อาจแตกต่างกัน)

การปรับแต่งฟอร์มด้วย Bootstrap

- ลักษณะการจัดวางโครงร่างของฟอร์ม เมื่อใช้ร่วมกับ Bootstrap จะเป็นดังภาพ



การปรับแต่งฟอร์มด้วย Bootstrap

react/app1/src/form-bootstrap.js

```
import React from 'react'

export default function FormBootstrap() {
  return (
    <div className="mt-3 mx-auto p-3" style={{ width:'400px', background:'#cee'}}>
      <form>
        <div className="form-group mb-3">
          <label htmlFor="login">Email</label>
          <input type="text" id="login" name="login" className="form-control"/>
        </div>
        <div className="form-group mb-3">
          <label htmlFor="pswd">Password</label>
          <input type="password" id="pswd" name="pswd" className="form-control"/>
        </div>
      </form>
    </div>
  )
}
```

การปรับแต่งฟอร์มด้วย Bootstrap

```
<div className="form-group mb-4">
  <label htmlFor="select1">Select</label>
  <select id="select1" name="select1" className="form-select">
    <option value="1">Item 1</option>
    <option value="2">Item 2</option>
    <option value="3">Item 3</option>
  </select>
</div>
<button className="btn btn-primary">OK</button>
</form>
</div>
)
```

```
}
```

การปรับแต่งฟอร์มด้วย Bootstrap

react/app1/src/index.js

```
import React from 'react';
import ReactDOM from 'react-dom';
import './index.css';
import App from './App';
import reportWebVitals from './reportWebVitals';
```

```
import 'bootstrap/dist/css/bootstrap.min.css'
import 'bootstrap/dist/js/bootstrap.bundle.min.js'
```

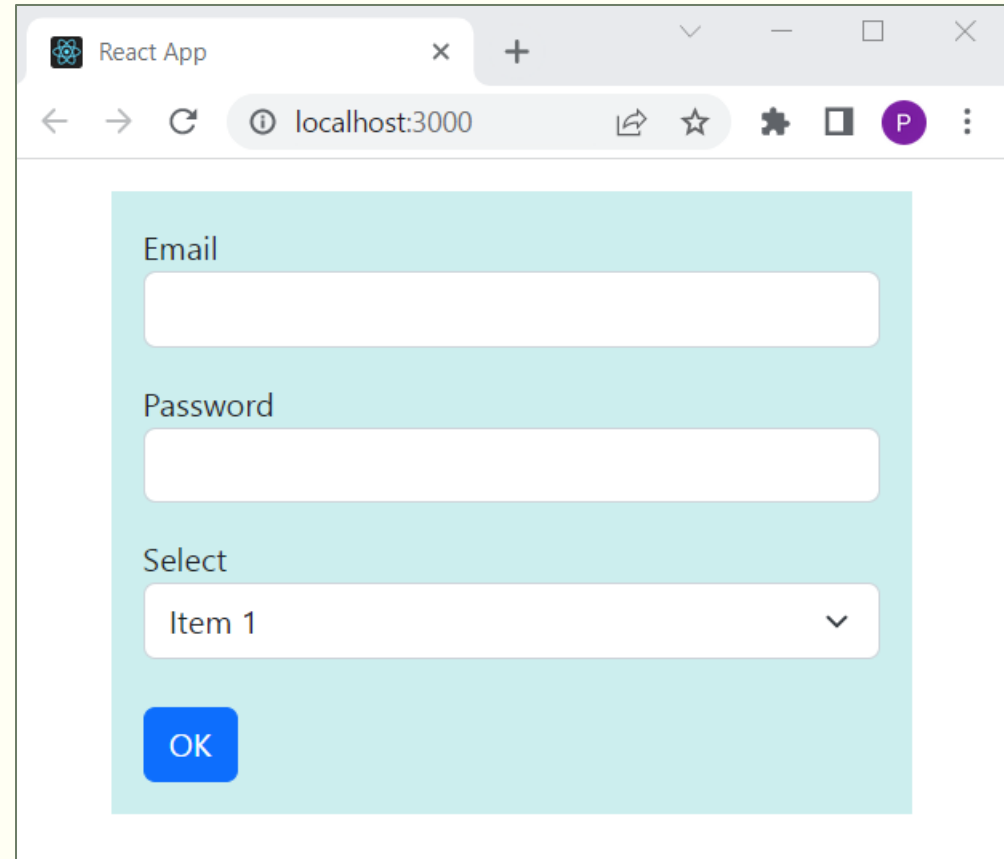
```
ReactDOM.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
  document.getElementById('root')
);
```

การปรับแต่งฟอร์มด้วย Bootstrap

react/app1/src/app.js

```
import React from 'react'
import FormBootstrap from './form-bootstrap'

export default function App() {
  return <FormBootstrap/>
}
```



React App

localhost:3000

Email

Password

Select

Item 1

OK

อินพุตชนิด text, password และ textarea

อินพุตชนิด text, password และ textarea จะมีวิธีดำเนินการที่คล้ายคลึงกัน ดังนั้น จึงขอกล่าวถึงวิธีการใช้งานร่วมกับ React ไปพร้อมๆ กัน โดยลักษณะแท็กและแอตทริบิวต์ที่สำคัญของอินพุตเหล่านี้คือ

```
<input type="text" id="id" name="ชื่อ" value="ค่าเริ่มแรก" size="ขนาด"
      placeholder="ข้อความลายน้ำ" maxlength="จำนวนอักขระสูงสุด">
```

```
<input type="password" แอตทริบิวต์อื่นๆ เช่นเดียวกับ text>
```

```
<textarea id="id" name="ชื่อ" rows="จำนวนแถว" cols="จำนวนบรรทัด"
      placeholder="ข้อความลายน้ำ">ค่าเริ่มต้น</textarea>
```

สำหรับแนวทางการใช้ร่วมกับ React และ Bootstrap มีดังนี้

- หากเราจำเป็นต้องเข้าถึงอินพุตในกลุ่มนี้ เช่น การอ่านค่าเพื่อนำไปตรวจสอบ หรือการกำหนดค่าตามเงื่อนไข ก็ต้องทำตามหลักการของ React นั่นคือ สร้างค่าอ้างอิงกับอินพุตด้วย `useRef()` หรือ `React.createRef()` เหมือนที่เราเคยทำกันมา

อินพุตชนิด `text`, `password` และ `textarea`

- การอ่านและกำหนดค่า ก็ใช้พร็อพเพอร์ตี้ซึ่งส่วนใหญ่ก็มีชื่อเดียวกับแอตทริบิวต์ เช่น

value	อ่านค่าที่ปรากฏอยู่บนอินพุตในขณะนั้น หรือกำหนดค่าใหม่ให้กับอินพุต ซึ่งค่าที่ได้จะเป็นชนิด String ดังนั้น เราสามารถใช้เมธอดของสตริงที่มีอยู่แล้วใน JavaScript เพื่อดำเนินการต่อไปได้
disabled	อยู่ในสถานะที่ใช้งานได้หรือไม่ โดยจะมีค่าเป็น true/false

- อินพุตในกลุ่มนี้ มีอีเวนต์ที่น่าสนใจ ซึ่งในบางครั้งเราจำเป็นต้องใช้เพื่อตอบสนองต่อเหตุการณ์บางอย่าง ดังนี้

input	เมื่อเปลี่ยนแปลงข้อความในอินพุต
focus	เมื่อย้ายโฟกัสมาที่อินพุตนั้น
blur	เมื่อย้ายโฟกัสออกไปจากอินพุตนั้น
keypress	เมื่อพิมพ์อักขระแต่ละตัวลงในอินพุต
keydown	เมื่อกดปุ่มคีย์บอร์ด (ตรวจสอบปุ่มได้)

อินพุตชนิด text, password และ textarea

- การปรับแต่งด้วย Bootstrap ก็มีคลาสที่เกี่ยวข้องโดยตรงดังนี้

form-group	กำหนดกลุ่มอินพุตและ label
form-label	กำหนดให้แก่อีลิเมนต์ที่ใช้เขียนข้อความกำกับอินพุต
form-control	กำหนดให้แก่อินพุต ในขนาดปกติ
form-control-sm	กำหนดให้แก่อินพุต เพื่อให้มีขนาดเล็กกว่าปกติ
form-control-lg	กำหนดให้แก่อินพุต เพื่อให้มีขนาดใหญ่กว่าปกติ ทั้งคลาส -sm และ -lg ต้องใช้ร่วมกับคลาส form-control

อินพุตชนิด text, password และ textarea

react/app1/src/form-text.js

```
import React from 'react'

export default function FormText() {
  const textPswd = React.useRef()
  const textPswd2 = .....

  const .....{
    let pswd = textPswd.current.value
    if (..... !== '' && !pswd.match(/^[0-9a-zA-Z]+$\/)) {
      textPswd.current.value = ''
      alert('.....')
    }
  }
}
```

```
const onBlurPassword2 = () => {
  if (.....){
    textPswd2.current.value = ''
    alert('รหัสผ่านทั้งสองช่องไม่ตรงกัน')
  }
}
```

อินพุตชนิด text, password และ textarea

react/app1/src/form-text.js (ต่อ)

```
return (  
  <div className="mt-4 mx-auto p-3 rounded"  
    style={{ width:'400px', background:'#cee'}}>  
    <form>  
      <div className="form-group mb-2">  
        <label htmlFor="login">ชื่อผู้ใช้ (Login)</label>  
        <input type="....." id="login" name="login" maxLength="20"  
          className="form-control form-control-sm"/>  
      </div>  
      <div className="form-group mb-2">  
        <label htmlFor="pswd">รหัสผ่าน</label>  
        <input type="....." id="pswd" name="pswd" maxLength="10"  
          className="form-control form-control-sm"  
          ref={.....} onBlur={onBlurPassword}/>  
      </div>  
    </div>  
  </div>  
)
```

อินพุตชนิด text, password และ textarea

react/app1/src/form-text.js (ต่อ)

```
<div className=".....">
  <label htmlFor="pswd2">.....</label>
  <input type="....." id="pswd2"
    className="form-control form-control-sm"
    ref={.....} onBlur={.....}/>
</div>
<div className="form-group mb-4">
  <label htmlFor="memo">.....</label>
  <textarea id="memo" name="memo" rows="2"
    maxLength="200"
    className=".....">
  </textarea>
</div>
```

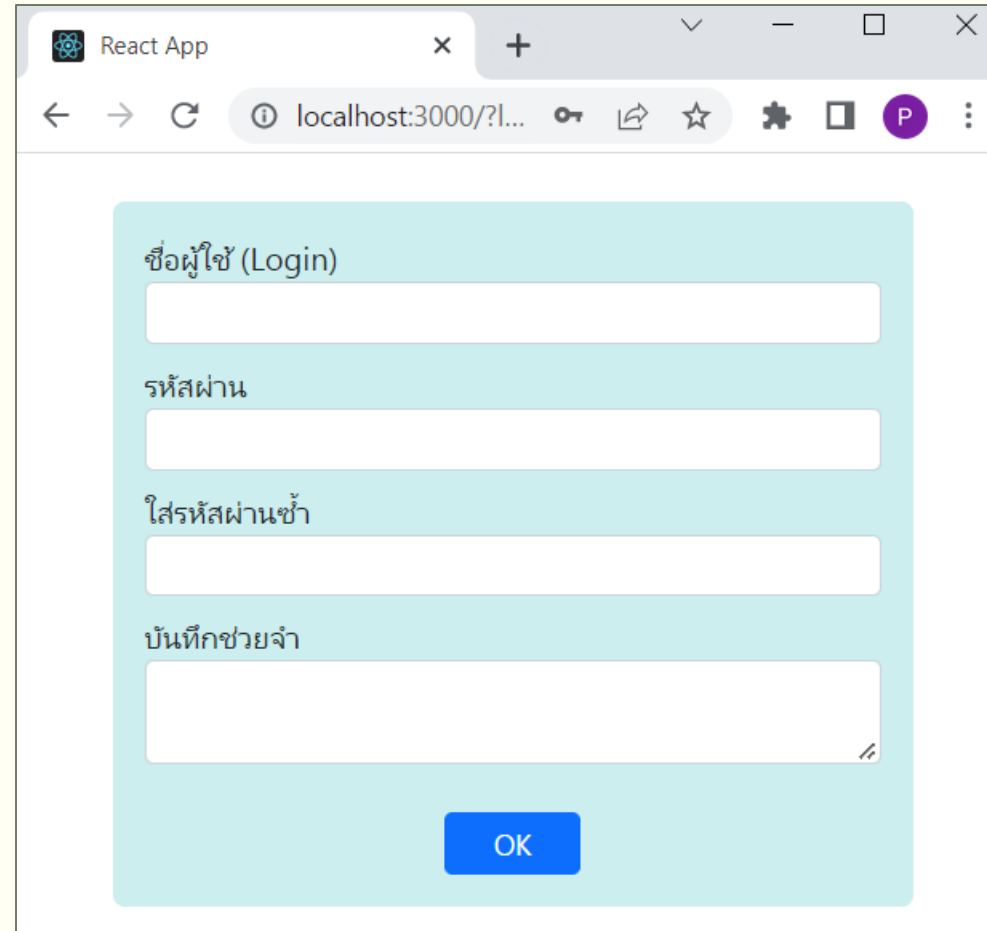
```
<div className="text-center">
  <button .....="btn btn-sm px-4 .....">
    OK
  </button>
</div>
</form>
</div>
)
}
```

อินพุตชนิด text, password และ textarea

react/app1/src/app.js

```
import ..... from 'react'
import FormText from .....

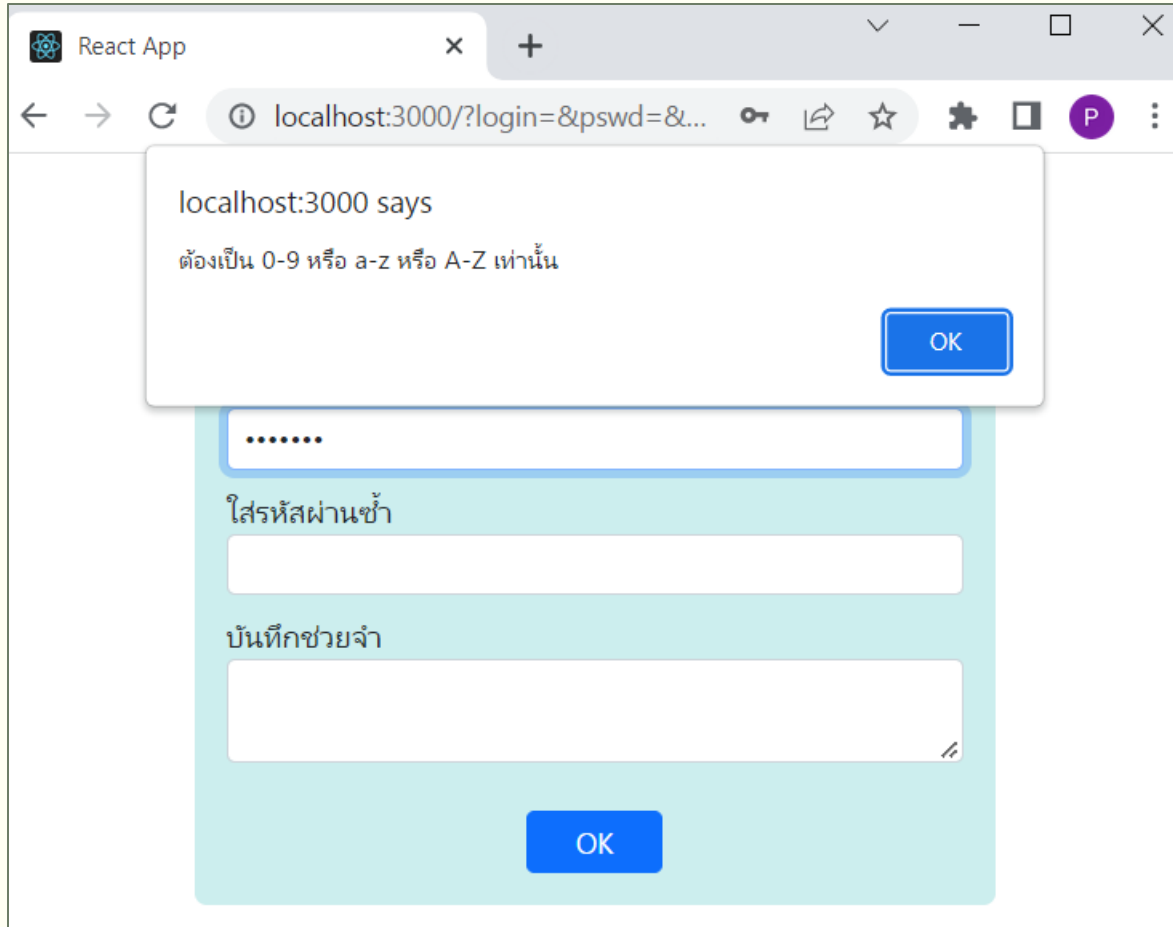
export default function App() {
  return <FormText/>
}
```



The screenshot shows a web browser window titled "React App" with the address bar displaying "localhost:3000/?l...". The browser window contains a light blue login form with the following elements:

- A label "ชื่อผู้ใช้ (Login)" above a text input field.
- A label "รหัสผ่าน" above a password input field.
- A label "ใส่รหัสผ่านซ้ำ" above a text input field.
- A label "บันทึกช่วยจำ" above a checkbox.
- A blue "OK" button at the bottom right of the form.

อินพุตชนิด text, password และ textarea



React App

localhost:3000/?login=&pswd=&...

localhost:3000 says
ต้องเป็น 0-9 หรือ a-z หรือ A-Z เท่านั้น

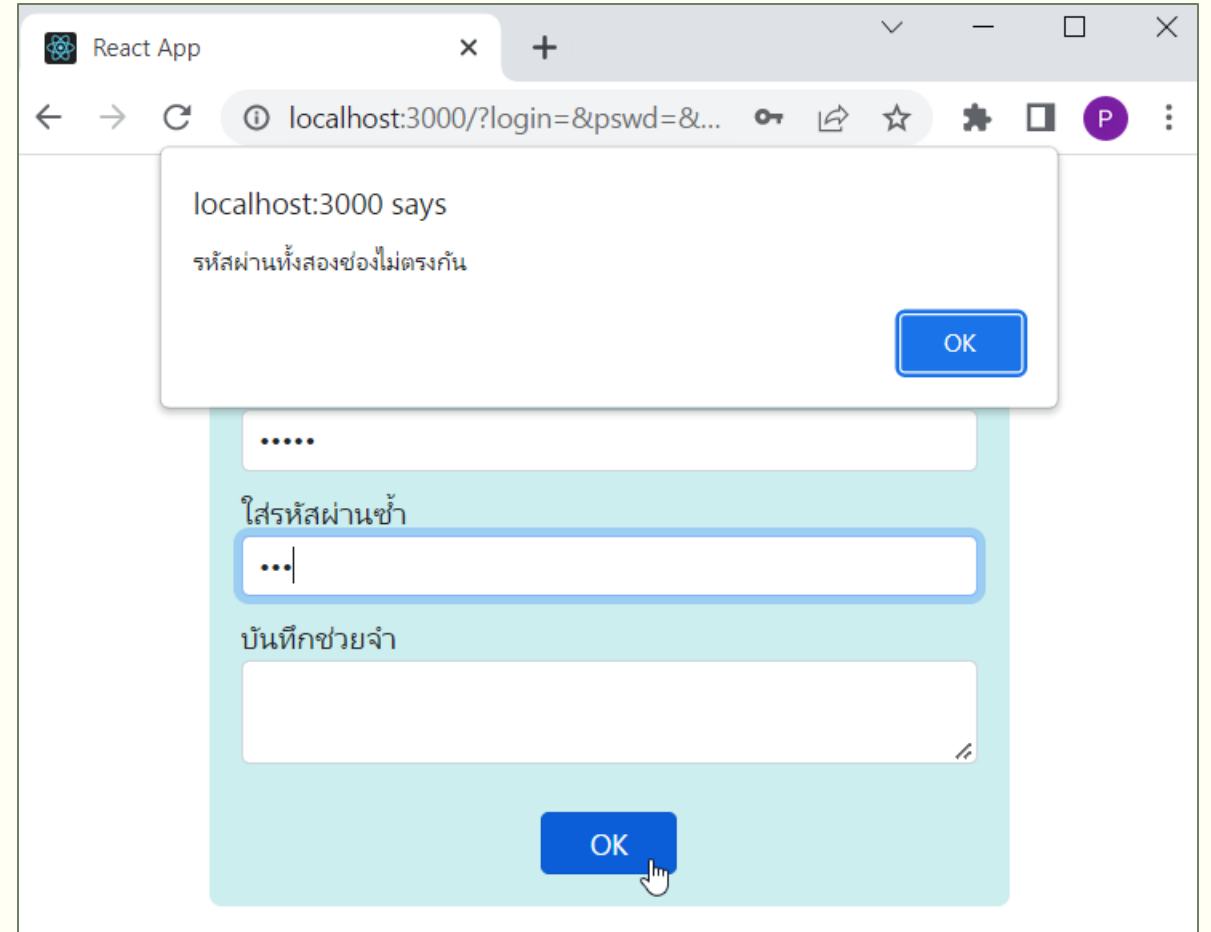
OK

.....

ใส่รหัสผ่านซ้ำ

บันทึกช่วยจำ

OK



React App

localhost:3000/?login=&pswd=&...

localhost:3000 says
รหัสผ่านทั้งสองช่องไม่ตรงกัน

OK

.....

ใส่รหัสผ่านซ้ำ

...

บันทึกช่วยจำ

OK

อินพุตชนิด select

อินพุตชนิด select ใช้สำหรับการเลือกรายการ โดยแบ่งออกเป็น 2 แบบคือ แบบที่เลือกเพียงรายการเดียวกับแบบที่เลือกได้มากกว่า 1 รายการ ซึ่งลักษณะของแท็กและแอตทริบิวต์ที่สำคัญมีดังนี้

```
<select id="id" name="ชื่อ" [multiple] [size="จำนวนรายการที่แสดงผล"]>
  <option value="ค่าของรายการที่ 1" [selected]>ข้อความบนรายการที่ 1</option>
  <option value="ค่าของรายการที่ 2" [selected]>ข้อความบนรายการที่ 2</option>
  ...
</select>
```

หากเราไม่กำหนดแอตทริบิวต์ multiple จะแสดงผลแบบ dropdown คือเลือกได้เพียงรายการเดียว แต่หากกำหนด multiple จะแสดงผลแบบ list ซึ่งสามารถเลือกได้มากกว่า 1 รายการ ส่วนแนวทางการใช้ร่วมกับ React และ Bootstrap มีดังนี้

- หากเราจำเป็นต้องเข้าถึงอินพุตชนิดนี้ เช่น การตรวจสอบสถานะของการเลือก และ/หรือ อ่านค่าไปใช้งาน ก็ต้องสร้างค่าอ้างอิงกับอินพุตด้วย useRef()

อินพุตชนิด select

- อินพุต select มีพร็อพเพอร์ตี้สำหรับการตรวจสอบและอ่านค่าที่น่าสนใจดังนี้

value	อ่านค่าแอตทริบิวต์ value ของรายการ (option) ที่ถูกเลือกในขณะนั้น
selectedIndex	ตรวจสอบลำดับรายการที่ถูกเลือก หรือกำหนดให้เลือกรายการลำดับนั้น
options[i].value	อ่านค่า (value) ของรายการลำดับที่ i
options[i].text	ข้อความบนรายการ (ระหว่าง <option> กับ </option>) ของรายการที่ i
options[i].selected	ตรวจสอบว่าได้เลือกรายการในลำดับที่ i หรือไม่
type	ชนิดรายการ ระหว่าง select-one กับ select-multiple
multiple	ตรวจสอบว่าได้ระบุแอตทริบิวต์ multiple หรือไม่ ถ้าระบุจะได้ค่าเป็น true

- หากต้องการตอบสนองทันทีที่เปลี่ยนแปลงการเลือกรายการ ให้กำหนดอีเวนต์ **change**
- การปรับแต่งด้วย Bootstrap สำหรับกรณีของ select ก็มีคลาสที่เกี่ยวข้องดังนี้

form-group	สำหรับกำหนดกลุ่มอินพุตและ label
form-select	กำหนดให้แก่ select เพื่อปรับแต่ง

อินพุตชนิด file

อีลิเมนต์ชนิด file ใช้สำหรับการเลือกไฟล์จากเครื่องของผู้ใช้ เพื่ออัปโหลดไปยังเซิร์ฟเวอร์ ซึ่งลักษณะของแท็กและแอตทริบิวต์ที่สำคัญมีดังนี้

```
<input type="file" id="id" name="ชื่อ" accept="ชนิดไฟล์ที่ยอมรับ" [multiple]>
```

สำหรับชนิดของไฟล์ที่ยอมรับ ให้กำหนดค่าในแบบ MIME Type เช่น video/* หมายถึงเลือกไฟล์วิดีโอได้ทุกประเภท หรือ image/png หมายถึงเลือกได้เฉพาะภาพชนิด png เป็นต้น ส่วน multiple จะระบุก็ต่อเมื่อเราอนุญาตให้เลือกมากกว่า 1 ไฟล์ ส่วนแนวทางการใช้ร่วมกับ React และ Bootstrap มีดังนี้

- หากเราจำเป็นต้องเข้าถึงอินพุตชนิดนี้ เช่น การตรวจสอบเกี่ยวกับไฟล์ ก็ให้สร้างค่าอ้างอิงของอินพุตด้วย useRef() หรือ createRef()

อินพุตชนิด file

- อินพุต file มีพร็อพเพอร์ตี้ที่น่าสนใจ ดังนี้

value	อ่านชื่อไฟล์ที่ถูกเลือกในขณะนั้น แต่ไม่สามารถกำหนดค่าได้ หรือกล่าวได้ว่า พร็อพเพอร์ตี้ value ของ file สามารถอ่านค่าได้อย่างเดียวเท่านั้น
files.length	จำนวนไฟล์ที่เลือก มักใช้ร่วมกับกรณีที่ระบุแอตทริบิวต์ multiple
files[i].name	ชื่อไฟล์ลำดับที่ i (เฉพาะชื่อ ไม่รวมตำแหน่งไฟล์)
files[i].size	ขนาดของไฟล์ลำดับที่ i (หน่วยเป็น byte)

- อีเวนต์สำคัญที่เกิดกับอินพุตชนิดนี้คือ **change** ซึ่งเกิดเมื่อเราเลือกไฟล์เสร็จแล้ว
- การปรับแต่งด้วย Bootstrap สำหรับกรณีของอินพุต file ก็มีคลาสที่เกี่ยวข้องดังนี้

form-group	กำหนดกลุ่มอินพุตและ label
form-label	กำหนดให้แก่อีลิเมนต์ที่ใช้เขียนข้อความกำกับอินพุต
form-control	กำหนดให้แก่อินพุต ในขนาดปกติ

อินพุตชนิด file

form-control-sm	กำหนดให้แก่อินพุต เพื่อให้มีขนาดเล็กกว่าปกติ
form-control-lg	กำหนดให้แก่อินพุต เพื่อให้มีขนาดใหญ่กว่าปกติ ทั้งคลาส -sm และ -lg ต้องใช้ร่วมกับคลาส form-control

ต่อไปเป็นแนวทางการทดสอบอินพุต file ร่วมกับ React ซึ่งในที่นี้จะนำ select มาใช้ร่วมด้วย เพื่อสร้างตัวเลือกจำนวนไฟล์และขนาดสูงสุดของไฟล์ แล้วนำไปเป็นเงื่อนไขในการตรวจสอบไฟล์ที่ถูกเลือกในอินพุตชนิด file มีดังนี้

อินพุตชนิด file

react/app1/src/form-select-file.js

```
import React from 'react'

export default function ..... {
  const inputFile = .....
  const selectMaxNumFiles = React.useRef()
  const ..... = React.useRef()

  let maxNumfiles = [1,2,3,4,5]           //จำนวนไฟล์ที่เลือกได้
  let ..... = [50, 100, 200, 500, 1000]  //ขนาดสูงสุดของไฟล์
```

อินพุตชนิด file

react/app1/src/form-select-file.js (ต่อ)

```
//เมื่อคลิกปุ่ม ให้นำค่าจาก select ทั้งสองอันมาเป็นเงื่อนไขตรวจสอบไฟล์
const onClickButton = () => {
  let maxNumFiles = selectMaxNumFiles.current.value
  let maxSize = .....
  //ตรวจสอบจำนวนไฟล์
  if (inputFile.current.files.length > .....){
    alert(`เลือกได้ไม่เกิน ${maxNumFiles} ไฟล์`)
    return
  }
}
```

```
//ตรวจสอบขนาดของแต่ละไฟล์
for (let f of inputFile.current.files) {
  if (f.size > maxSize * 1000) {
    alert(`.....`)
    return
  }
}
alert('Files OK')
}
```

อินพุตชนิด file

react/app1/src/form-select-file.js (ต่อ)

```
return (  
  <div className="mt-4 ..... p-3 rounded"  
    style={{ width:'400px', background:'#cee'}}>  
  
    <form>  
      <div className="form-group mb-3">  
        <label htmlFor="maxNumFiles" className="form-label">จำนวนไฟล์สูงสุด</label>  
        <select id="....." className="form-select form-select-sm" ref={.....}>  
          {  
            maxNumfiles.map(i => {  
              return <..... value={i}>{.....}</.....>  
            })  
          }  
        </select>  
      </div>  
    </div>  
  )
```

อินพุตชนิด file

react/app1/src/form-select-file.js (ต่อ)

```
<div className="form-group mb-3">
  <label htmlFor="....." className="form-label">ขนาดไฟล์สูงสุด (KB)</label>
  <select id="maxFileSize" className="form-select form-select-sm" ref={selectMaxSize}>
    {
      .....(i => {
        return <option value={i}>{i}</option>
      })
    }
  </select>
</div>
```

อินพุตชนิด file

react/app1/src/form-select-file.js (ต่อ)

```
<div className="..... mb-3">
  <label htmlFor="file" className=".....">เลือกไฟล์</label>
  <input type="....." id="file" accept="image/*"
    className="..... form-control-sm" ref={.....} multiple/>
</div>

<div className="text-center mt-4">
  <button type="button" className="btn btn-sm ..... px-4"
    onClick={.....}>OK
  </button>
</div>
</form>
</div>
)
```

```
}
```

อินพุตชนิด file

react/app1/src/app.js

```
import React from 'react'
import FormSelectFile from './form-select-file';

export default function App() {
  return <FormSelectFile/>
}
```

React App

localhost:3000

จำนวนไฟล์สูงสุด

1

ขนาดไฟล์สูงสุด (KB)

50

เลือกไฟล์

Choose Files No file chosen

OK

React App

localhost:3000

localhost:3000 says
ขนาดของแต่ละไฟล์ต้องไม่เกิน 50 KB

OK

ขนาดไฟล์สูงสุด (KB)

50

เลือกไฟล์

Choose Files RPReplay_Final1664613262.MP4

OK

React App

localhost:3000

localhost:3000 says
เลือกได้ไม่เกิน 1 ไฟล์

OK

ขนาดไฟล์สูงสุด (KB)

50

เลือกไฟล์

Choose Files 2 files

OK